



WHZ Westsächsische
Hochschule Zwickau
Hochschule für Mobilität

Echtzeitbetriebsysteme

Semesterprojekt

Radar-Sensor

Teil Softwarereglung

Name: Fan, Jau-Wen Lagger, Simon Englmaier, Michael
Seminargruppe: 222702
Datum: 29.06.2025

Inhaltsverzeichnis

1	Mathematische Theroie	1
1.1	Oszilloskop importieren	1
1.2	Filterauswahl	2
1.2.1	Filterauswahlmöglichkeit – FIR	2
1.2.2	Filterauswahlmöglichkeit – IIR	3
1.3	Spektrumanalyse	4
1.4	Hauptfrequenz finden	5
1.5	IIR-Spektrum-Visualisierung	6
2	ESP32-Implementierung	10
2.1	Radarmodul (Slave)	11
2.1.1	Mathematische Methode in C	11
2.1.2	Daten auf CAN verpacken	12
2.1.3	CAN-Kommunizierung	13
2.1.4	Vollständiger Code	14
2.2	ECU (Master)	18
2.2.1	CAN-Kommunizierung	18
2.2.2	Vollständiger Code	19

Abbildungsverzeichnis

1.1	Plot Quellsignal	6
1.2	Plot FIR-Fliter	7
1.3	Plot IIR-Fliter	7
1.4	Plot Spektrumanalyse	8
1.5	Plot Hauptfrequenz	8
1.6	Plot IIR-Spektrum-Visualisierung	9
2.1	CAN-Protokoll	10

Tabellenverzeichnis

2.1	CAN-Protokoll, Hexzahl	12
2.2	Slave-Steuerprozess	13

Mathematische Theroie

Dieses Kapitel erklärt die Mathematik der analogen Signale, die digital verarbeitet und in CAN-Pakete verpackt werden. Vor der Übertragung an CAN wird das analoge Quellsignal zunächst mit einem Analog-Digital-Wandler(ADC) in ein digitales Signal umgewandelt, durch einen digitalen Tiefpassfilter geleitet und dann einer Fast-Fourier-Transformation(FFT) unterzogen, um die dominante Frequenz zu bestimmen, die dann in die Geschwindigkeit in km/h umgerechnet wird.

1.1 Oszilloskop importieren

Das Radarsignal wird zuert im MatLab importieren. Das Plot(Abb. 1.1) zeigt, dass in der Quellsignal viele Grate (Hochfrequenzrauschen) vorhanden sind.

```
1  %-----1.Read wave from oscillator-----%
   data = readmatrix('WFM02.CSV');
   voltage = data(1:end, 2); % jump header

   %-----2.Plot-----%
6  time = data(1:end, 1);      % first col. = time
   figure;
   plot(time, voltage);
   title('Raw-Plot');
   xlabel('Time (Sec)');
11  ylabel('Voltage (V)');
```

Listing 1.1. Oszilloskop importieren

1.2 Filterauswahl

Es gibt zwei Algorithmusmethode von Digitalfiltern, Filter mit endlicher Impulsantwort (FIR) und Filter mit unendlicher Impulsantwort (IIR).

1.2.1 Filterauswahlmöglichkeit – FIR

In diesem Abschnitt werden die Ergebnisse des FIR-Algorithmus vorgestellt. Übertragungsfunktion:

$$G(z) = \frac{y(z)}{u(z)} = b_0 + b_1 z^{-1} + \dots + b_M z^{-M}$$

Eigenschaften:

- immer stabil, toleranter bzgl. Rundung von Koeffizienten
- benötigen ggf. hohe Filterordnung (in diesem Fall 6000. Ordnung)¹

Ergebnis im Abb. 1.2 gezeigt.

```

%-----3.1FIR Filter-----%
% FIR design(Lowpass 1kHz, fs=??Hz)
Ts_FIR = 0.23999304 / 130432; %time elapse/data count
4 % Data space is not equidistant
fs_FIR = 1 / Ts_FIR; %freq. from oscillator
f_cut_FIR = 1150;
order = 6000;
h_FIR = fir1(order, f_cut_FIR/(fs_FIR/2), 'low', hamming(order +
9     1));

% apply FIR
voltage_filtered = filter(h_FIR, 1, voltage);

% plot bode
14 fvtool(h_FIR, 1, 'Fs', fs_FIR)

Ts = 1/fs_FIR;
Hz = tf(h_FIR, 1, Ts); % z-transform
% coeff. draw
19 num = Hz.Numerator{1}; % Numerator = h_FIR

%visualization
figure;
plot(time, voltage_filtered);
24 title('FIR-Filtered-Plot');
xlabel('Time (Sec)');
ylabel('Voltage (V)');

%Overlay Comparison
29 figure;
plot(time, voltage, 'b', 'DisplayName', 'Raw'); % original(blue)
hold on;
plot(time, voltage_filtered, 'r', 'DisplayName', 'FIR-Filtered'); %
    FIR(red)
hold off;
34

```

¹ Herbst, Digitale Signalverarbeitung, DSV V-05-2

```

39 title('Raw vs. Filtered Signal');
xlabel('Time (Sec)');
ylabel('Voltage (V)');
legend('show');
grid on;

```

Listing 1.2. FIR Filter

1.2.2 Filterauswahlmöglichkeit – IIR

In diesem Abschnitt werden die Ergebnisse des IIR-Algorithmus (Butterworth) vorgestellt. Übertragungsfunktion:

$$G(z) = \frac{y(z)}{u(z)} = \frac{b_0 + b_1 z^{-1} + \dots + b_{(M-1)} z^{-(M-1)}}{1 + a_1 z^{-1} + \dots + a_M z^{-M}}$$

Eigenschaften:

- kein Ripple im Durchlassbereich
 - unabhängig von Filterordnung immer 3 dB Dämpfung bei Grenzfrequenz²
- Ergebnis im Abb. 1.3 gezeigt. Bei diesem Algorithmus erzeugt es weniger Phasenverschiebung, gleichzeitig liegt es weniger Last auf Mikroprocessor.

```

1  %-----3.2IIR Filter-----%
   fs_IIR = fs_FIR;
   f_cut_IIR = 1150; % cut freq.
   order_IIR = 6;

6  % design Butterworth IIR
   [b, a] = butter(order_IIR, f_cut_IIR/(fs_IIR/2), 'low');

   %Bode
   fvtool(b, a, 'Fs', fs_IIR);

11 % apply IIR
   voltage_filtered_IIR = filter(b, a, voltage);

   %visualization
16 figure;
   plot(time, voltage_filtered_IIR);
   title('IIR-Filtered-Plot');
   xlabel('Time (Sec)');
   ylabel('Voltage (V)');

21 %overlap compare
   figure;
   plot(time, voltage, 'b', 'DisplayName', 'Raw'); % original(blue)
   hold on;
26 plot(time, voltage_filtered_IIR, 'm', 'DisplayName',
        'IIR-Filtered'); % apply IIR(red)
   hold off;

   title('Raw vs. IIR-Filtered Signal');
   xlabel('Time (Sec)');

```

² Herbst, Digitale Signalverarbeitung, DSV V-04-13

```

31     ylabel('Voltage (V)');
    legend('show');
    grid on;

```

Listing 1.3. IIR Filter

1.3 Spektrumanalyse

Aus dem Datenblatt des Radars³ geht hervor, dass die Geschwindigkeit proportional zur Frequenz des Signals ist. Daher ist es notwendig, das Signal in Frequenzbereich umzuwandeln⁴. Obwohl das Signal-Rausch-Verhältnis des IIR-Filters niedriger als FIR ist, reichen 60 dB aus, um das Signal eindeutig zu identifizieren. Ergebnis im Abb. 1.4 gezeigt.

```

%-----4.Spectrum analyze-----%
2
% Filter Spectrum analyze:before vs after
% Remove DC(both original and filtered)
voltage_nofilter = voltage - mean(voltage);
voltage_filtered = voltage_filtered - mean(voltage_filtered);
7
% coeff. setup
fs = fs_FIR;
N = length(voltage); % same length

12 % original FFT
window1 = hamming(N);
signal1 = voltage_nofilter .* window1;
fft1 = fft(signal1);
fft1 = fft1(1:floor(N/2)+1);
17 fft1_db = 20*log10(abs(fft1) / max(abs(fft1)));

% FIR filtered FFT
window2 = hamming(N);
signal2 = voltage_filtered .* window2;
22 fft2 = fft(signal2);
fft2 = fft2(1:floor(N/2)+1);
fft2_db = 20*log10(abs(fft2) / max(abs(fft2)));

% IIR filtered FFT
27 window3 = hamming(N);
signal3 = voltage_filtered_IIR .* window3;
fft3 = fft(signal3);
fft3 = fft3(1:floor(N/2)+1);
fft3_db = 20*log10(abs(fft3) / max(abs(fft3)));
32
% freq. axis
freq = (0:floor(N/2))' * fs / N;

% visualization
37 figure;
plot(freq, fft1_db, 'b', 'DisplayName', 'Raw');

```

³ RSM-1650

⁴ Understanding FFTs and Windowing


```

42     hold on;
        plot(freq, fft2_db, 'r', 'DisplayName', 'FIR-Filtered');
        plot(freq, fft3_db, 'm', 'DisplayName', 'IIR-Filtered');
        title('FFT: Raw vs. Filtered (0 dB normalized)');
        xlabel('Frequenz (Hz)');
        ylabel('Amplitude (dB)');
        legend('show');
        xlim([0 5000]); % until 5kHz
47     grid on;

```

Listing 1.4. Spektrumanalyse

1.4 Hauptfrequenz finden

Die Verstärkungskompensation für das RC-Filter vor dem ADC-Eingang wird erst am Anfang dieses Abschnitts hinzugefügt. Ergebnis im Abb. 1.5 gezeigt.

```

3     %-----5.find main freq.-----%
        % analog gain compensation
        % RC 1. Order
        f_c = 1000; % f_-3dB
        gain_comp = sqrt(1 + (freq / f_c).^2);

8     % compensated FFT amplitude
        fft3_corrected = abs(fft3) .* gain_comp;

        % normalize and than transfer to dB
        fft3_corrected_db = 20*log10(fft3_corrected / max(fft3_corrected));

13    %fprintf('Main Freq.: %.2f Hz, Amplitude: %.6f V\n', peak_freq,
        amplitude_corrected);
        %mark on plot
        fft_magnitude = abs(fft3);
        fft_magnitude_db = fft3_corrected_db;
        [peak_val, peak_idx] = max(fft_magnitude);
18    peak_freq = freq(peak_idx);
        peak_db = fft_magnitude_db(peak_idx);

        % visualization
        figure;
23    plot(freq, fft_magnitude_db, 'm');
        hold on;
        xline(peak_freq, 'r--');
        text(peak_freq, peak_db, sprintf('%.1f Hz\n(%.1f dB)', peak_freq,
            peak_db), ...
            'Color', 'red', 'VerticalAlignment', 'top', 'FontSize', 10);

28    title('IIR, FFT, Hamming, normalized');
        xlabel('Frequency (Hz)');
        ylabel('Amplitude (dB)');
        xlim([0 5000]);
33    grid on;
        legend('FFT Signal');

```

Listing 1.5. Hauptfrequenz

1.5 IIR-Spektrum-Visualisierung

Wenn später neue Programmversion das gesamte Spektrum unterhalb von -40 dB entfernt wird, können auch die Hauptfrequenz und die Amplitude ermittelt werden. Ergebnis wie Abb. 1.6 gezeigt.

```

1      %----6.IIR Spectrum visualization----%
      % exist: fft3 = fft(signal3);
      %fft3_abs = abs(fft3); % same as above
      %N = length(voltage); % same as above
      amplitude = 2 * fft3_corrected(peak_idx) / N;
6      window_gain = mean(hamming(N));
      amplitude_corrected = amplitude / window_gain;

      fprintf('Main Freq. %.2f Hz, Amplitude %.6f V\n', peak_freq,
              amplitude_corrected);

11     fft3_voltage = abs(fft3) / N * 2 / window_gain;
      figure;
      bar(freq, fft3_voltage, 'FaceColor', [0.4 0.6 0.8], 'EdgeColor',
          'k'); % custom color
      title('IIR-Filtered Signal Spectrum (Bar)');
      xlabel('Frequency (Hz)');
16     ylabel('Amplitude (V)');
      xlim([0 1000]); % 1kHz
      grid on;

```

Listing 1.6. IIR-Spektrum-Visualisierung

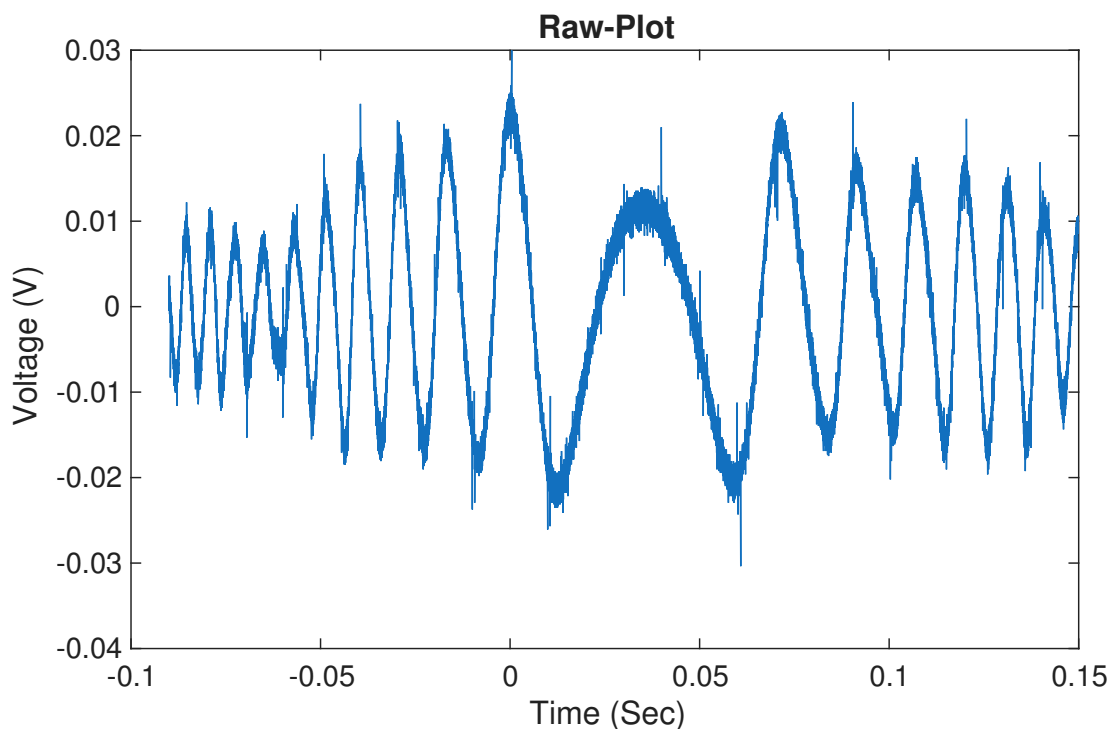


Abbildung 1.1. Plot Quellsignal

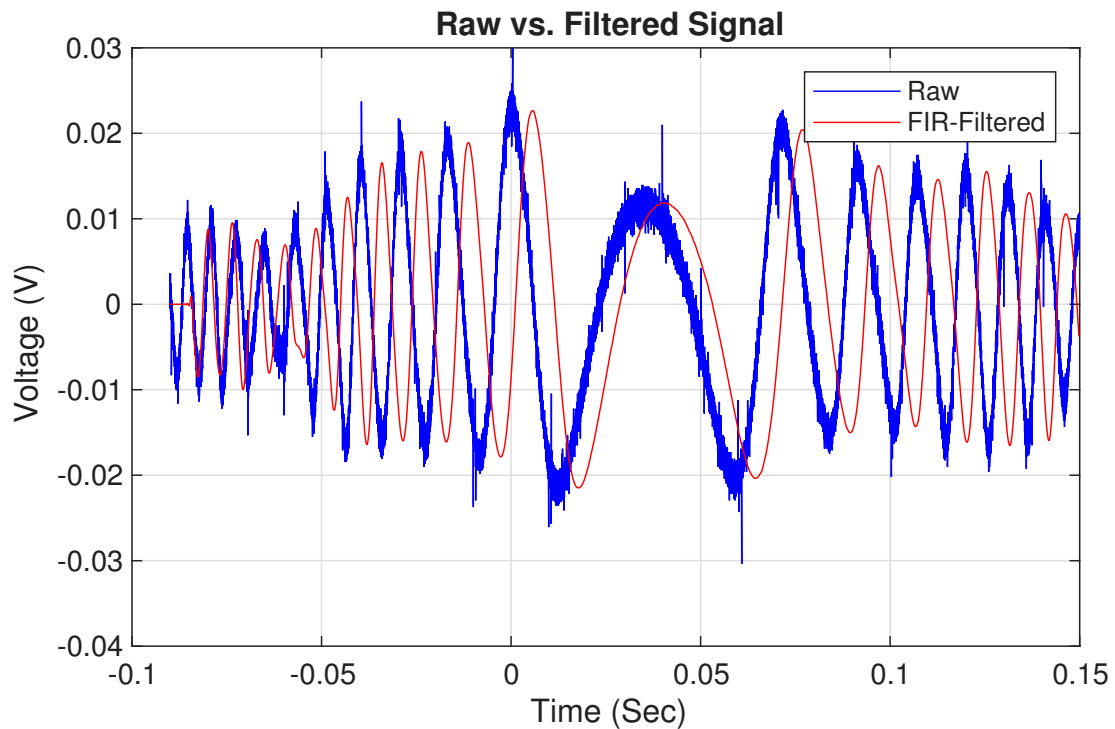


Abbildung 1.2. Plot FIR-Fliter: Das Rauschen wird merklich unterdrückt und erzeugt zudem eine sichtbare Phasenverschiebung. (6000.te Ordnung)

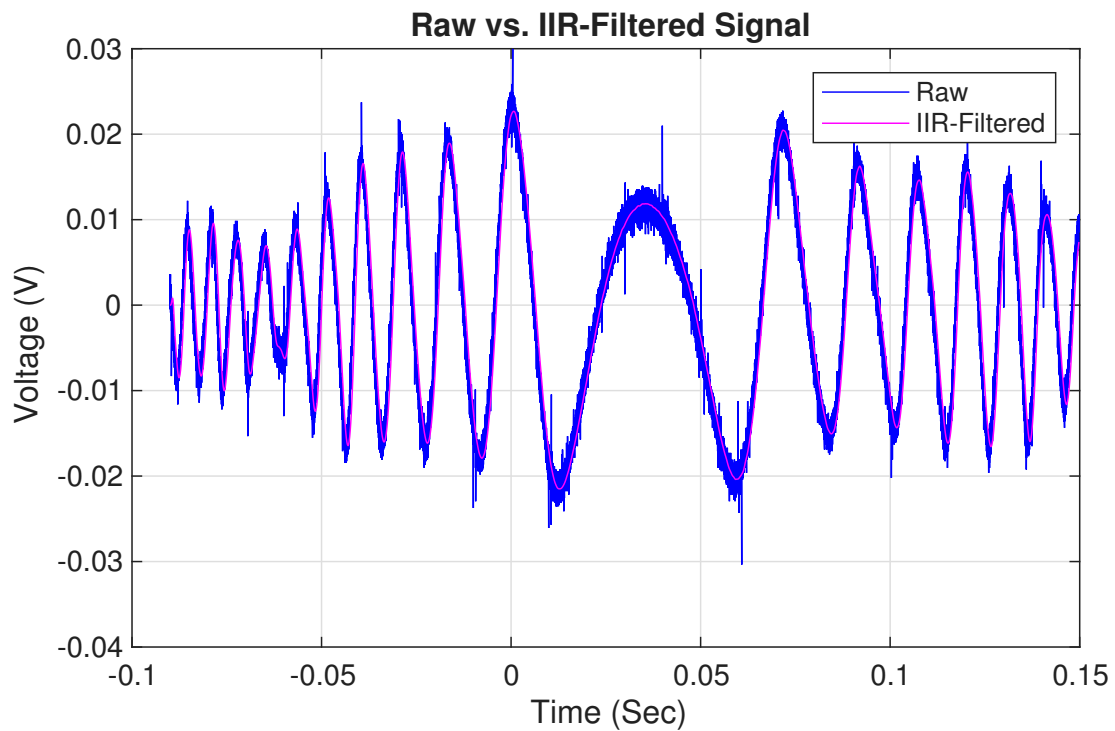


Abbildung 1.3. Plot IIR-Fliter: Das Rauschen wird auch merklich unterdrückt aber erzeugt kaum sichtbare Phasenverschiebung. (6.te Ordnung)

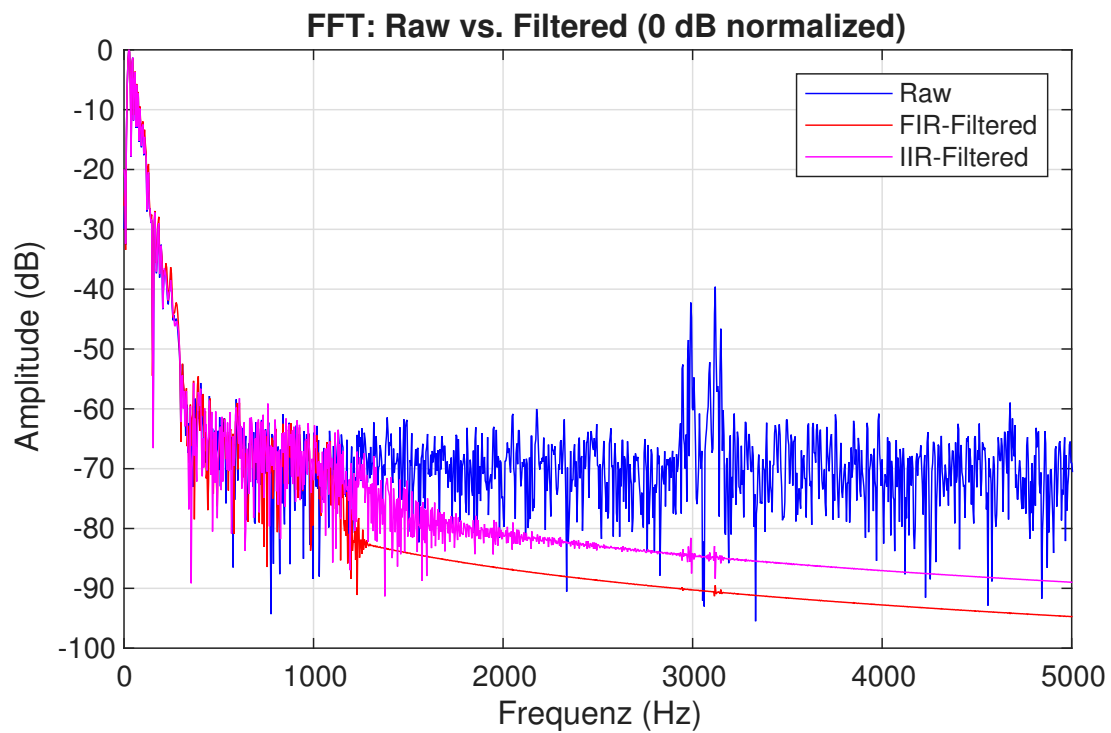


Abbildung 1.4. Plot Spektrumanalyse

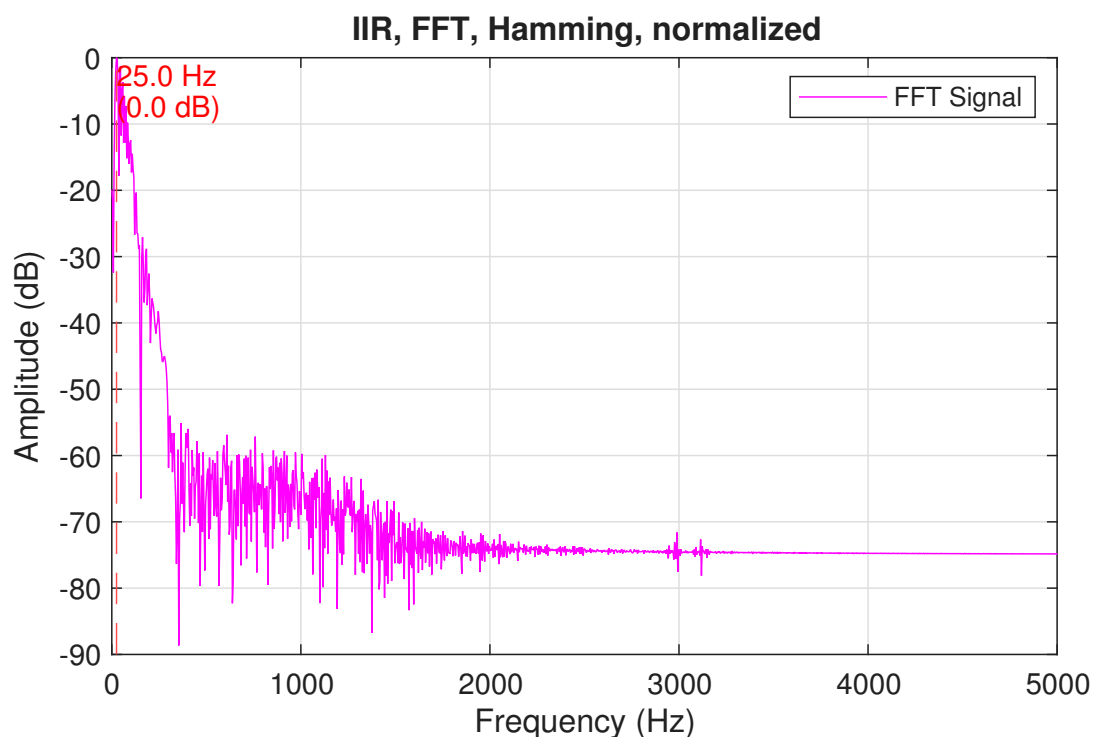


Abbildung 1.5. Plot Hauptfrequenz

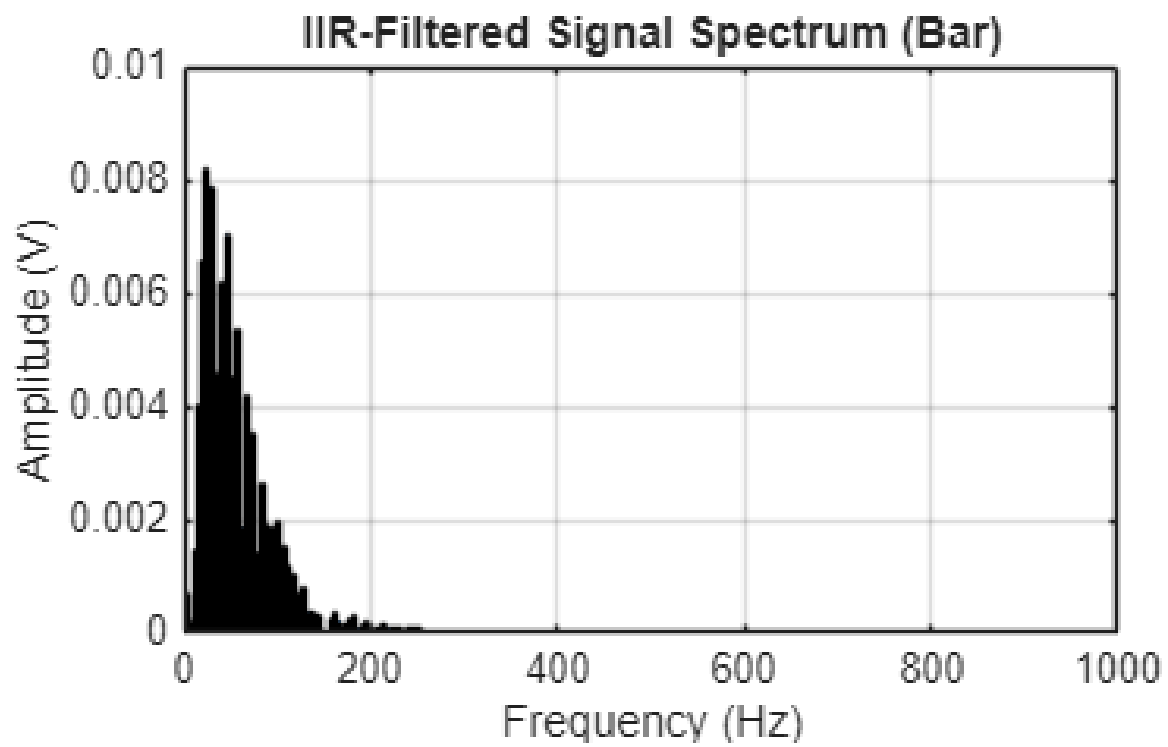


Abbildung 1.6. Plot IIR-Spektrum-Visualisierung

ESP32-Implementierung

Dieses Projekt entwickelt ein Kfz-Radarsensormodul und sendet Signale über CAN. Dieses Kapitel wendet die mathematische Theorie auf den ESP32-Mikroprozessor (μC) an. Das Fahrzeugradar ist der Slave im Fahrzeug-CAN-System, und es gibt einen weiteren ESP32-Master- μC im Projekt, um das Zentralsteuergerät (ECU) im Fahrzeug zu simulieren.¹

CAN Data Link Layer: Data Frame

- Aufbau eines CAN-Data-Frames:

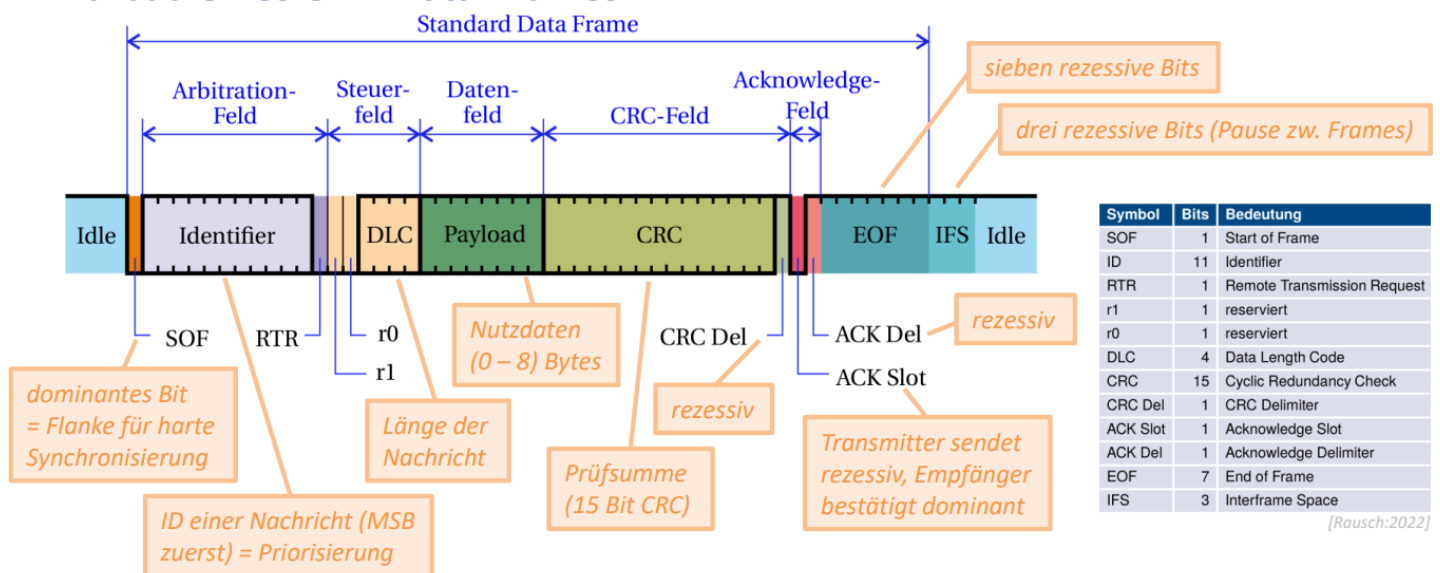


Abbildung 2.1. CAN-Protokoll

¹ Herbst, Fahrzeug-Kommunikationssysteme, FKS V-04-21

2.1 Radarmodul (Slave)

Die gesamte Salve-Aufgabe wird in mehrere Funktionen aufgeteilt, zwei Kerne des ESP32, Core0 ist für die Berechnung von Signalen und Core1 für das Senden und Empfangen von Daten sowie für die Überwachung zuständig.

2.1.1 Mathematische Methode in C

Mathematische Operationen unter direkter Verwendung von Bibliotheken.

```
#include <math.h>
2 #include "kiss_FFT.h"
```

Listing 2.1. Bibliothek in C für Mathe

Konvertiert Matlab-Algorithmen in C-Code und speichert die Ergebnisse in der Strukturvariable **item**, wobei die Geschwindigkeit und die Spannung aufgezeichnet werden.

```
static void radar_task(void *arg)
{
3   ESP_ERROR_CHECK(i2s_driver_install(I2S_NUM_0, &i2s_cfg, 0, NULL));
   ESP_ERROR_CHECK(i2s_set_adc_mode(ADC_UNIT_1, ADC1_CHANNEL_4)); // GPIO32

   adc1_config_width(ADC_WIDTH_BIT_12);
   adc1_config_channel_atten(ADC1_CHANNEL_4, ADC_ATTEN_DB_11);
8   ESP_ERROR_CHECK(i2s_adc_enable(I2S_NUM_0));

   uint16_t raw_buf[FFT_LEN];
   float    filt_buf[FFT_LEN];
   float    fft_buf[2 * FFT_LEN];
13  float    y_prev = 0;

   dsps_fft2r_init_fc32(NULL, CONFIG_DSP_MAX_FFT_SIZE);

   while (true) {
18     size_t n_read = 0;
     ESP_ERROR_CHECK(i2s_read(I2S_NUM_0, raw_buf, sizeof(raw_buf), &n_read,
                             portMAX_DELAY));
     if (n_read < sizeof(raw_buf)) continue;

     /* IIR LPF */
23     for (int i = 0; i < FFT_LEN; i++) {
         float x = (float)(raw_buf[i] >> 4);
         y_prev = y_prev + alpha * (x - y_prev);
         filt_buf[i] = y_prev;
     }
28     /* Build complex array */
     for (int i = 0; i < FFT_LEN; i++) {
         fft_buf[2 * i] = filt_buf[i];
         fft_buf[2 * i + 1] = 0.0f;
     }
33     dsps_fft2r_fc32(fft_buf, FFT_LEN);
     dsps_bit_rev_fc32(fft_buf, FFT_LEN);
     dsps_cplx2reC_fc32(fft_buf, FFT_LEN);

     float max_mag = 0.0f; int max_bin = 1;
38     for (int i = 1; i < FFT_LEN / 2; i++) {
         float re = fft_buf[2 * i], im = fft_buf[2 * i + 1];
         float mag = sqrtf(re * re + im * im);
         if (mag > max_mag) { max_mag = mag; max_bin = i; }
     }
}
```

```

43     float freq_hz = ((float)max_bin * SAMPLE_RATE_HZ) / FFT_LEN;
    float speed_kph = freq_hz / SPEED_SCALE;
    float amp_adc = 2.0f * max_mag / FFT_LEN;
    float volt_peak = amp_adc * ADC_V_PER_COUNT;

48     radar_data_t item = {.speed_kph = speed_kph, .volt_peak = volt_peak};
    xQueueSend(radar_q, &item, portMAX_DELAY);
    }
}

```

Listing 2.2. Mathematische Methode in C

2.1.2 Daten auf CAN verpacken

Datenspeicherplatz

[0][1] sind Geschwindigkeit gespeichert.

[2][3] sind Amplitude(Spannung) gespeichert.

Variable **item** wird folgende DATA[0...3] gespeichert und auf CAN-Protokoll verpackt (Hex-Wert, **0x**Unterlasst):

(Bsp.: 440Hz, 1,25V input)

Identifier	DLC	DATA[0]	DATA[1]	DATA[2]	DATA[3]
556	4	03	E8	04	E2

Tabelle 2.1. CAN-Protokoll, Hexzahl

```

static void can_tx_task(void *arg)
{
    twai_message_t data_msg = {
4      .extd = 0, .rtr = 0, .ss = 0, .self = 0,
        .identifier = ID_SLAVE_DATA,
        .data_length_code = 4,
    };

9      while (true) {
        /* Block until streaming is enabled */
        if (!stream_enabled) {
            xSemaphoreTake(start_sem, portMAX_DELAY);
        }
14     radar_data_t data;
        if (xQueueReceive(radar_q, &data, portMAX_DELAY) == pdPASS) {
            if (!stream_enabled) continue; // started/stopped during wait
            uint16_t s_fixed = (uint16_t)(data.speed_kph * 100.0f + 0.5f);
            uint16_t v_fixed = (uint16_t)(data.volt_peak * 1000.0f + 0.5f);
19     data_msg.data[0] = s_fixed >> 8;
            data_msg.data[1] = s_fixed & 0xFF;
            data_msg.data[2] = v_fixed >> 8;
            data_msg.data[3] = v_fixed & 0xFF;
            twai_transmit(&data_msg, pdMS_TO_TICKS(5));
24     /* ----- Console output ----- */
            ESP_LOGI(TAG,
                "TX Speed %.2f km/h, Vpk %.3f V (raw 0x%04X 0x%04X)",
                data.speed_kph, data.volt_peak, s_fixed, v_fixed);
        }
29     }
}

```


Listing 2.3. Daten auf CAN verpacken**2.1.3 CAN-Kommunizierung**

Der Slave in CAN wartet nach dem Booten auf das Senden der Anrufe durch den Master. Das Prozess ist wie folgt:

Befehl	Variable	HEX-ID
Master-Anrufe	ID_MASTER_PING	0xA2
Slave-Antwort	ID_SLAVE_PING_RESP	0xB2
Master-Start	ID_MASTER_START_CMD	0xA1
Signal Luft	...	...
Master-Stopp	ID_MASTER_STOP_CMD	0xA0
Slave-Stopp	ID_SLAVE_STOP_CMD	0xB0

Tabelle 2.2. Slave-Steuerprozess

```

static void can_rx_task(void *arg)
{
    twai_message_t rx;
    twai_message_t resp_ping = {
5      .extd = 0, .rtr = 0, .ss = 0, .self = 0,
      .identifier = ID_SLAVE_PING_RESP, .data_length_code = 0 };
    twai_message_t resp_stop = {
      .extd = 0, .rtr = 0, .ss = 0, .self = 0,
10     .identifier = ID_SLAVE_STOP_RESP, .data_length_code = 0 };

    while (true) {
        if (twai_receive(&rx, portMAX_DELAY) != ESP_OK) continue;
        switch (rx.identifier) {
            case ID_MASTER_PING:
15             twai_transmit(&resp_ping, pdMS_TO_TICKS(5));
             break;
            case ID_MASTER_START_CMD:
                stream_enabled = true;
                xSemaphoreGive(start_sem);
20             ESP_LOGI(TAG, "Start-CMD received -> streaming ON");
             break;
            case ID_MASTER_STOP_CMD:
                stream_enabled = false;
                twai_transmit(&resp_stop, pdMS_TO_TICKS(5));
25             xSemaphoreGive(stop_sem);
             ESP_LOGI(TAG, "Stop-CMD received -> streaming OFF");
             break;
            default:
                break;
30         }
    }
}

```

Listing 2.4. Slave-Steuerprozess

Slave-Monitor zeigt folgende Infos:

```

2      Start-CMD received -> streaming ON
      ...
      TX Speed 10.00 km/h, Vpk 1.250 V (raw 0x03E8 0x04E2)
4      ...
      Stop-CMD received -> streaming OFF

```

Listing 2.5. Slave-Monitor, Komunizierung

2.1.4 Vollständiger Code

```

// twai_slave_radar_main.c - Dual-core radar DSP + CAN (TWAI) slave
// -----
// Flow compatible with the original TWAI Master demo:
// 1) Master sends 0x0A2 (PING) -> Slave replies 0x0B2 (PING-RESP)
5 // 2) Master sends 0x0A1 (START) -> Slave starts Tx 0x556 (DATA)
// 3) Master sends 0x0A0 (STOP) -> Slave stops + Tx 0x0B0 (STOP-RESP)
//
// - Core 0: ADC (I2S-DMA) -> IIR -> FFT -> speed + peak voltage
// - Core 1: Rx-task handles master commands, Tx-task streams data when
// enabled
10 // Ctrl-task monitors Bus-off.
//
// Build: ESP-IDF v5.x (classic ESP32)
// -----

15 #include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
20 #include "freertos/queue.h"
#include "freertos/semphr.h"
#include "esp_err.h"
#include "esp_log.h"
#include "driver/adc.h"
25 #include "driver/i2s.h"
#include "driver/twai.h"
#include "esp_dsp.h"

/* -----
*/
30 /* Board / Application config
*/
/* -----
*/

// #ifndef CONFIG_EXAMPLE_TX_GPIO_NUM
#define CONFIG_EXAMPLE_TX_GPIO_NUM 16 // <- edit in menuconfig if required
// #endif
35 // #ifndef CONFIG_EXAMPLE_RX_GPIO_NUM
#define CONFIG_EXAMPLE_RX_GPIO_NUM 17
// #endif

#define TAG "RADAR_CAN_SLAVE"

40 /* --- TWAI (CAN)
----- */

#define TWAI_TX_TASK_PRIO 8
#define TWAI_RX_TASK_PRIO 8
#define TWAI_CTRL_TASK_PRIO 10
45

/* Master <-> Slave command IDs (11-bit standard) */
#define ID_MASTER_STOP_CMD 0x0A0
#define ID_MASTER_START_CMD 0x0A1
#define ID_MASTER_PING 0x0A2

```

```

50  #define ID_SLAVE_STOP_RESP    0x0B0
    #define ID_SLAVE_DATA        0x556    // <speed, volt> payload
    #define ID_SLAVE_PING_RESP   0x0B2

    static const twai_timing_config_t t_cfg = TWAI_TIMING_CONFIG_500KBITS();
55  static const twai_filter_config_t f_cfg = TWAI_FILTER_CONFIG_ACCEPT_ALL();
    static const twai_general_config_t g_cfg =
        TWAI_GENERAL_CONFIG_DEFAULT(CONFIG_EXAMPLE_TX_GPIO_NUM,
        CONFIG_EXAMPLE_RX_GPIO_NUM,
        TWAI_MODE_NORMAL);    // real bus <-> master

60  /* --- DSP constants
    ----- */

    #define SAMPLE_RATE_HZ    8000
    #define FFT_LEN            1024
    #define FC_HZ              1000    // IIR 1-pole cutoff (Hz)
65  #define SPEED_SCALE         44.0f    // freq->speed (44 Hz per km/h)
    static const float alpha = (2.0f * M_PI * FC_HZ) / ((float)SAMPLE_RATE_HZ +
        2.0f * M_PI * FC_HZ);
    /* ADC -> voltage conversion (ADC1 12-bit, 11 dB atten ~ 0-3.6 V) */
    #define ADC_MAX_COUNTS    4095.0f
    #define ADC_V_PER_COUNT   (3.6f / ADC_MAX_COUNTS)

70  /* --- I2S (ADC-DMA) configuration
    ----- */

    static const i2s_config_t i2s_cfg = {
        .mode = I2S_MODE_MASTER | I2S_MODE_RX | I2S_MODE_ADC_BUILT_IN,
        .sample_rate = SAMPLE_RATE_HZ,
75  .bits_per_sample = 16,
        .communication_format = I2S_COMM_FORMAT_I2S_MSB,
        .channel_format = I2S_CHANNEL_FMT_ONLY_RIGHT,
        .intr_alloc_flags = ESP_INTR_FLAG_LEVEL1,
        .dma_buf_count = 4,
        .dma_buf_len = 256,
        .use_apll = false,
        .tx_desc_auto_clear = false,
        .fixed_mclk = 0
80  };

85  /* -----
    */
    /* FreeRTOS objects
    */
    /* -----
    */

90  typedef struct {
        float speed_kph;
        float volt_peak;    // in Volts
    } radar_data_t;

    static QueueHandle_t radar_q;    // DSP -> Tx
95  static SemaphoreHandle_t start_sem; // Rx-task -> Tx-task (start)
    static SemaphoreHandle_t stop_sem; // Rx-task -> Tx-task (stop)

    static volatile bool stream_enabled = false;    // Tx-task gate

100  /* -----
    */
    /* Core 0 - Radar DSP
    */
    /* -----
    */

    static void radar_task(void *arg)
105  {
        ESP_ERROR_CHECK(i2s_driver_install(I2S_NUM_0, &i2s_cfg, 0, NULL));
        ESP_ERROR_CHECK(i2s_set_adc_mode(ADC_UNIT_1, ADC1_CHANNEL_4));    // GPIO32

        adc1_config_width(ADC_WIDTH_BIT_12);
        adc1_config_channel_atten(ADC1_CHANNEL_4, ADC_ATTEN_DB_11);

```

```

110     ESP_ERROR_CHECK(i2s_adc_enable(I2S_NUM_0));

    uint16_t raw_buf[FFT_LEN];
    float    filt_buf[FFT_LEN];
    float    fft_buf[2 * FFT_LEN];
115     float    y_prev = 0;

    dsps_fft2r_init_fc32(NULL, CONFIG_DSP_MAX_FFT_SIZE);

    while (true) {
        size_t n_read = 0;
        ESP_ERROR_CHECK(i2s_read(I2S_NUM_0, raw_buf, sizeof(raw_buf), &n_read,
                                portMAX_DELAY));
        if (n_read < sizeof(raw_buf)) continue;

        /* IIR LPF */
125     for (int i = 0; i < FFT_LEN; i++) {
        float x = (float)(raw_buf[i] >> 4);
        y_prev = y_prev + alpha * (x - y_prev);
        filt_buf[i] = y_prev;
    }

    /* Build complex array */
130     for (int i = 0; i < FFT_LEN; i++) {
        fft_buf[2 * i] = filt_buf[i];
        fft_buf[2 * i + 1] = 0.0f;
    }

135     dsps_fft2r_fc32(fft_buf, FFT_LEN);
    dsps_bit_rev_fc32(fft_buf, FFT_LEN);
    dsps_cplx2reC_fc32(fft_buf, FFT_LEN);

    float max_mag = 0.0f; int max_bin = 1;
140     for (int i = 1; i < FFT_LEN / 2; i++) {
        float re = fft_buf[2 * i], im = fft_buf[2 * i + 1];
        float mag = sqrtf(re * re + im * im);
        if (mag > max_mag) { max_mag = mag; max_bin = i; }
    }

145     float freq_hz = ((float)max_bin * SAMPLE_RATE_HZ) / FFT_LEN;
    float speed_kph = freq_hz / SPEED_SCALE;
    float amp_adc = 2.0f * max_mag / FFT_LEN;
    float volt_peak = amp_adc * ADC_V_PER_COUNT;

150     radar_data_t item = {.speed_kph = speed_kph, .volt_peak = volt_peak};
    xQueueSend(radar_q, &item, portMAX_DELAY);
    }
}

155     /* -----
        */
    /*                      Core 1 - CAN Tx & Rx
        */
    /* -----
        */

    static void can_tx_task(void *arg)
    {
160         twai_message_t data_msg = {
            .extd = 0, .rtr = 0, .ss = 0, .self = 0,
            .identifier = ID_SLAVE_DATA,
            .data_length_code = 4,
        };

165         while (true) {
            /* Block until streaming is enabled */
            if (!stream_enabled) {
                xSemaphoreTake(start_sem, portMAX_DELAY);
            }

170             radar_data_t data;
            if (xQueueReceive(radar_q, &data, portMAX_DELAY) == pdPASS) {
                if (!stream_enabled) continue; // started/stopped during wait
            }
        }
    }
}

```

```

175         uint16_t s_fixed = (uint16_t)(data.speed_kph * 100.0f + 0.5f);
        uint16_t v_fixed = (uint16_t)(data.volt_peak * 1000.0f + 0.5f);
        data_msg.data[0] = s_fixed >> 8;
        data_msg.data[1] = s_fixed & 0xFF;
        data_msg.data[2] = v_fixed >> 8;
        data_msg.data[3] = v_fixed & 0xFF;
180         twai_transmit(&data_msg, pdMS_TO_TICKS(5));
        /* ----- Console output ----- */
        ESP_LOGI(TAG,
185         "TX Speed %.2f km/h, Vpk %.3f V (raw 0x%04X 0x%04X)",
        data.speed_kph, data.volt_peak, s_fixed, v_fixed);
    }
}

static void can_rx_task(void *arg)
190 {
    twai_message_t rx;
    twai_message_t resp_ping = {
        .extd = 0, .rtr = 0, .ss = 0, .self = 0,
        .identifier = ID_SLAVE_PING_RESP, .data_length_code = 0 };
195    twai_message_t resp_stop = {
        .extd = 0, .rtr = 0, .ss = 0, .self = 0,
        .identifier = ID_SLAVE_STOP_RESP, .data_length_code = 0 };

    while (true) {
200        if (twai_receive(&rx, portMAX_DELAY) != ESP_OK) continue;
        switch (rx.identifier) {
            case ID_MASTER_PING:
                twai_transmit(&resp_ping, pdMS_TO_TICKS(5));
                break;
205            case ID_MASTER_START_CMD:
                stream_enabled = true;
                xSemaphoreGive(start_sem);
                ESP_LOGI(TAG, "Start-CMD received -> streaming ON");
                break;
210            case ID_MASTER_STOP_CMD:
                stream_enabled = false;
                twai_transmit(&resp_stop, pdMS_TO_TICKS(5));
                xSemaphoreGive(stop_sem);
                ESP_LOGI(TAG, "Stop-CMD received -> streaming OFF");
215                break;
            default:
                break;
        }
    }
220 }

/* -----
    */
/*                               Core 1 - Bus monitor
    */
/* -----
    */

225 static void can_ctrl_task(void *arg)
{
    ESP_ERROR_CHECK(twai_start());
    ESP_LOGI(TAG, "TWAI driver started - waiting for master...");
    while (true) {
230        twai_status_info_t st; twai_get_status_info(&st);
        if (st.state == TWAI_STATE_BUS_OFF) {
            ESP_LOGW(TAG, "Bus-off detected - trying recovery");
            twai_initiate_recovery();
        }
235        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

```

```

240      /* -----
      /*
      /*
      /* -----
      /*
void app_main(void)
{
    ESP_ERROR_CHECK(twai_driver_install(&g_cfg, &t_cfg, &f_cfg));
245    radar_q  = xQueueCreate(4, sizeof(radar_data_t));
    start_sem = xSemaphoreCreateBinary();
    stop_sem  = xSemaphoreCreateBinary();

250    /* Core 0 - radar DSP */
    xTaskCreatePinnedToCore(radar_task, "radar", 4096*2, NULL, 9, NULL,
        PRO_CPU_NUM);

    /* Core 1 - CAN tasks */
    xTaskCreatePinnedToCore(can_tx_task, "can_tx", 4096, NULL,
        TWAI_TX_TASK_PRIO, NULL, APP_CPU_NUM);
255    xTaskCreatePinnedToCore(can_rx_task, "can_rx", 4096, NULL,
        TWAI_RX_TASK_PRIO, NULL, APP_CPU_NUM);
    xTaskCreatePinnedToCore(can_ctrl_task, "can_ctl", 2048, NULL,
        TWAI_CTRL_TASK_PRIO, NULL, APP_CPU_NUM);
}

```

Listing 2.6. Radarmodul, CAN-Slave

2.2 ECU (Master)

In diesem Projekt simuliert der Master das Steuergerät, um Radarsignale zu empfangen, und demonstriert, dass der Master den Befehl gibt, dass Slave Daten senden. Nach dem Empfang des ersten Signals beginnt Master, eine Minute zu zählen. Wenn die Zeit abgelaufen ist, sendet er den Befehl, die Übertragung an den Slave zu beenden.

2.2.1 CAN-Kommunizierung

Master-Monitor zeigt folgende Infos:

```

Driver started
2   Sending ping...
   Got ping-resp!
4   Sending start-cmd...
   First data received, starting 60 s window...
6   ...
   RX Speed 10.00 km/h, Vpk 1.250 V (raw 0x03E8 0x04E2)
8   RX Speed 9.92 km/h, Vpk 1.241 V (raw 0x03E0 0x04D9)
   ...
10  Time's up! Sending stop-cmd.
   Master exiting
12  Driver stopped
   Driver uninstalled

```

Listing 2.7. Master-Monitor, Komunizierung

2.2.2 Vollständiger Code

```

// twai_master_radar_main.c - Master node that parses <speed, voltage> 16-bit
// payload
2 // -----
// * Pings the radar-CAN *slave* (ID 0x556) and sends start/stop commands
// * Receives 4-byte data frames: [speed_MSB, speed_LSB, volt_MSB, volt_LSB]
//   -> speed_fixed = speed x 100 (0.01 km/h / LSB)
//   -> volt_fixed = Vpk x 1000 (1 mV / LSB)
7 //   Converts them back to floating-point and prints a nicely formatted line.
//
//   Tested with ESP-IDF v5.4 on classic ESP32. Bus speed set to 500 kbit/s to
//   match the *slave* example.
// -----
12
// Streamlined 1-minute test on first frame
// -----
// * Master pings Slave -> waits for PING_RESP
// * Immediately sends START command
17 // * On receiving first 0x556 frame, starts 1-minute timer
// * After 60 seconds, sends STOP command + exits
//
// Output is printed to monitor at each step
// -----
22
#include <stdio.h>
#include <stdlib.h>
#include <inttypes.h>
#include "freertos/FreeRTOS.h"
27 #include "freertos/task.h"
#include "freertos/queue.h"
#include "freertos/semphr.h"
#include "esp_err.h"
#include "esp_log.h"
32 #include "driver/twai.h"
#include "esp_timer.h"

#define TAG "TWAI Master"

37 #define CONFIG_EXAMPLE_TX_GPIO_NUM 16
#define CONFIG_EXAMPLE_RX_GPIO_NUM 17

#define ID_MASTER_STOP_CMD      0x0A0
#define ID_MASTER_START_CMD     0x0A1
42 #define ID_MASTER_PING        0x0A2
#define ID_SLAVE_STOP_RESP     0x0B0
#define ID_SLAVE_DATA          0x556
#define ID_SLAVE_PING_RESP     0x0B2

47 static const twai_general_config_t g_config =
TWAI_GENERAL_CONFIG_DEFAULT(CONFIG_EXAMPLE_TX_GPIO_NUM,
CONFIG_EXAMPLE_RX_GPIO_NUM,
TWAI_MODE_NORMAL);
static const twai_timing_config_t t_config = TWAI_TIMING_CONFIG_500KBITS();
52 static const twai_filter_config_t f_config = TWAI_FILTER_CONFIG_ACCEPT_ALL();

static QueueHandle_t tx_task_queue;
static QueueHandle_t rx_task_queue;
static SemaphoreHandle_t done_sem;
57

typedef enum {
    TX_SEND_PING,
    TX_SEND_START_CMD,
    TX_SEND_STOP_CMD,
62    TX_TASK_EXIT,
} tx_action_t;

typedef enum {

```

```

67     RX_WAIT_PING_RESP,
        RX_HANDLE_DATA,
        RX_WAIT_STOP_RESP,
        RX_TASK_EXIT,
    } rx_action_t;

72     /*-----Tx task-----*/
    static void twai_tx_task(void *arg) {
        while (1) {
            tx_action_t act;
            xQueueReceive(tx_task_queue, &act, portMAX_DELAY);

77            twai_message_t msg = {
                .extd = 0, .rtr = 0, .ss = 0, .self = 0, .dlc_non_comp = 0, .
                data_length_code = 0
            };

82            if (act == TX_SEND_PING) {
                ESP_LOGI(TAG, "Sending ping...");
                msg.identifier = ID_MASTER_PING;
                msg.ss = 1;
                twai_transmit(&msg, portMAX_DELAY);
87            } else if (act == TX_SEND_START_CMD) {
                ESP_LOGI(TAG, "Sending start-cmd...");
                msg.identifier = ID_MASTER_START_CMD;
                twai_transmit(&msg, portMAX_DELAY);
            } else if (act == TX_SEND_STOP_CMD) {
92                ESP_LOGI(TAG, "Time's up! Sending stop-cmd.");
                msg.identifier = ID_MASTER_STOP_CMD;
                twai_transmit(&msg, portMAX_DELAY);
            } else if (act == TX_TASK_EXIT) {
                break;
97            }
        }
        vTaskDelete(NULL);
    }

102    /*-----Rx task-----*/
    static void twai_rx_task(void *arg) {
        rx_action_t act;
        xQueueReceive(rx_task_queue, &act, portMAX_DELAY);
        bool timer_started = false;
107        int64_t t_start = 0;

        while (1) {
            twai_message_t rx_msg;
            if (twai_receive(&rx_msg, pdMS_TO_TICKS(500)) != ESP_OK) continue;

112            if (act == RX_WAIT_PING_RESP && rx_msg.identifier == ID_SLAVE_PING_RESP)
            {
                ESP_LOGI(TAG, "Got ping-resp!");
                tx_action_t next = TX_SEND_START_CMD;
                rx_action_t next_rx = RX_HANDLE_DATA;
117                xQueueSend(tx_task_queue, &next, 0);
                xQueueSend(rx_task_queue, &next_rx, 0);
                break;
            }
        }

122        while (1) {
            twai_message_t rx_msg;
            if (twai_receive(&rx_msg, pdMS_TO_TICKS(500)) != ESP_OK) continue;

127            if (rx_msg.identifier == ID_SLAVE_DATA && rx_msg.data_length_code == 4)
            {
                uint16_t s = (rx_msg.data[0] << 8) | rx_msg.data[1];
                uint16_t v = (rx_msg.data[2] << 8) | rx_msg.data[3];
                float spd = s / 100.0f, vp = v / 1000.0f;

```



```

    ESP_LOGI(TAG, "RX Speed %.2f km/h, Vpk %.3f V (raw 0x%04X 0x%04X)",
        spd, vp, s, v);
132
    if (!timer_started) {
        ESP_LOGI(TAG, "First data received, starting 60 s window...");
        timer_started = true;
        t_start = esp_timer_get_time();
137    }

    //Countdown timer for 60 seconds
    if (timer_started && esp_timer_get_time() - t_start > 600000000) {
        tx_action_t stop = TX_SEND_STOP_CMD;
142        tx_action_t quit = TX_TASK_EXIT;
        rx_action_t quit_rx = RX_TASK_EXIT;
        xQueueSend(tx_task_queue, &stop, 0);
        xQueueSend(tx_task_queue, &quit, 0);
        xQueueSend(rx_task_queue, &quit_rx, 0);
147        ESP_LOGI(TAG, "Master exiting.");
        xSemaphoreGive(done_sem);
        break;
    }
    else if (rx_msg.identifier == ID_SLAVE_STOP_RESP) {
152        ESP_LOGI(TAG, "Got stop-resp");
        break;
    }
}
vTaskDelete(NULL);
157 }

void app_main(void) {
    tx_task_queue = xQueueCreate(4, sizeof(tx_action_t));
    rx_task_queue = xQueueCreate(4, sizeof(rx_action_t));
162    done_sem = xSemaphoreCreateBinary();

    xTaskCreatePinnedToCore(twai_tx_task, "tx", 4096, NULL, 9, NULL, 1);
    xTaskCreatePinnedToCore(twai_rx_task, "rx", 4096, NULL, 8, NULL, 1);

167    ESP_ERROR_CHECK(twai_driver_install(&g_config, &t_config, &f_config));
    ESP_ERROR_CHECK(twai_start());
    ESP_LOGI(TAG, "Driver started");

    tx_action_t ping = TX_SEND_PING;
172    rx_action_t rxp = RX_WAIT_PING_RESP;
    xQueueSend(tx_task_queue, &ping, 0);
    xQueueSend(rx_task_queue, &rxp, 0);

    xSemaphoreTake(done_sem, portMAX_DELAY);
177    ESP_ERROR_CHECK(twai_stop());
    ESP_LOGI(TAG, "Driver stopped");
    ESP_ERROR_CHECK(twai_driver_uninstall());
    ESP_LOGI(TAG, "Driver uninstalled");
}

```

Listing 2.8. ECU CAN-Master